

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»

Кафедра систем управління літальними апаратами

Практична робота № 3

з дисципліни «Об'єктно-орієнтоване проєктування СУ»

Тема: «Структурування програм з використанням функцій»

ХАІ.301 . 172. 526. 10 ПР

Виконав студент гр. _____526_____

_____Пермінов Герман_____

(підпис, дата)

(П.І.Б.)

Перевірила

_____к.т.н., доц. О. В. Гавриленко

(підпис, дата)

(П.І.Б.)

2025

МЕТА РОБОТИ

Вивчити теоретичний матеріал із синтаксису визначення і виклику функцій та особливостей послідовностей у Python, а також документацію бібліотеки `numpy`; отримати навички реалізації бібліотеки функцій з параметрами, що структурують вирішення завдань «згори – до низу».

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. Описати функцію відповідно до варіанту. Для виклику функції (друга частина задачі) описати іншу функцію, що на вході має список вхідних даних і повертає список вихідних даних. Введення даних, виклик функції та виведення результатів реалізувати в третій функції без параметрів. Завдання наведено в табл.1.

Завдання 2. Розробити дві вкладені функції для вирішення задачі обробки двовимірних масивів відповідно до варіанту: зовнішня – без параметрів, внутрішня має на вході ім'я файлу з даними, на виході – підраховані параметри матриці (перша частина задачі) та перетворену матрицю (друга частина задачі). Для обробки масивів використати функції бібліотеки `numpy`. Завдання представлено в табл.2.

Завдання 3. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

- 1) *«Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»*
- 2) *«Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де*

ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

Проаналізувати задані питання, коментарі і оцінки, надані ШІ. Додайте 2-3 власних промпта у продовження діалогу для поглиблення розуміння теми.

ВИКОНАННЯ РОБОТИ

Завдання 1

Алгоритми вирішення показані нижче



Рисунок а – Task1_standart



Рисунок 6 – digit_count_list

Завдання 2

Алгоритм вирішення показано нижче

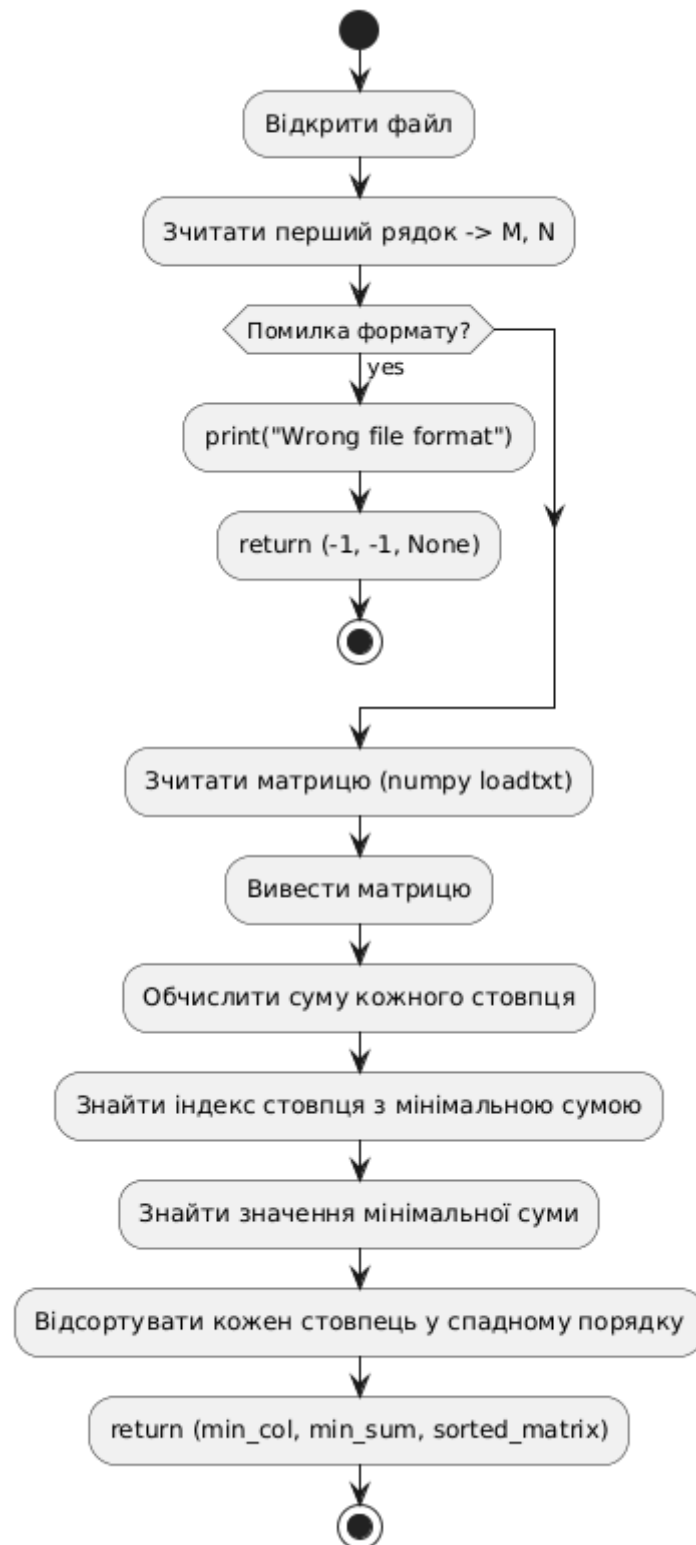


Рисунок в – matrix

ВИСНОВКИ

У роботі реалізовано функції для обробки списків і матриць: підрахунок кількості цифр у числах та опрацювання матриці з використанням NumPy. Виконано пошук стовпця з мінімальною сумою та сортування стовпців. Завдання виконані коректно.

ДОДАТОК А

lab3:

```
# підключення необхідної бібліотеки
import numpy as np

# ----- ЗАВДАННЯ 1 -----

def digit_count(K):
    """Повертає кількість цифр додатного числа K"""
    return len(str(abs(K)))

def digit_count_list(list_of_k):
    """Функція для обробки списку чисел відповідно до функції
за варіантом"""
    out_data = []
    for k in list_of_k:      # для кожного числа у вхідному
списку
        out_data.append(digit_count(k))
    return out_data

def task1_standart():
    """Стандартна версія обробки: цикл + append"""
    in_data = []
    try:
        in_num = int(input("Введіть кількість елементів: "))
        for i in range(in_num):
            temp = int(input(f"{i+1} element: "))
            in_data.append(temp)
    except ValueError:
```



```

        print("Помилка введення!")
    else:
        print("Кількість цифр:", digit_count_list(in_data))

def task1_list_comprehension():
    """Обробка з використанням list comprehension"""
    try:
        in_num = int(input("Введіть кількість елементів: "))
        in_data = [int(input(f"{i+1} element: ")) for i in
range(in_num)]
        except ValueError:
            print("Помилка введення!")
    else:
        out_data = [digit_count(k) for k in in_data]
        print("Кількість цифр:", out_data)

def task1_lambda():
    """Обробка списку за допомогою lambda + map"""
    try:
        in_data = list(map(lambda e: int(e), input("Enter
numbers: ").split()))
        except ValueError:
            print("Помилка введення!")
    else:
        out_data = list(map(lambda k: digit_count(k),
in_data))
        print("Кількість цифр:", out_data)

# ----- ЗАВДАННЯ 2 (Matrix10) -----

```

```

def matrix10(filename):
    """
    Зчитування матриці розміру M×N з файлу.
    1) Знаходить номер стовпця з мінімальною сумою
    2) Повертає номер стовпця, мінімальну суму та
        матрицю, відсортовану по кожному стовпцю у спадному
        порядку.
    """

    with open(filename, 'r') as f:
        # перший рядок: M N
        first_line = f.readline().split()
        try:
            M = int(first_line[0])
            N = int(first_line[1])
        except (IndexError, ValueError):
            print("Wrong file format!")
            return -1, -1, None

        # зчитування матриці
        matrix = np.loadtxt(filename, skiprows=1, max_rows=M)
        print("Вхідна матриця:")
        print(matrix)

        # 1) знайти суму кожного стовпця
        col_sums = np.sum(matrix, axis=0)

        # індекс стовпця з мінімальною сумою
        min_col_index = int(np.argmin(col_sums)) + 1 # +1 щоб
        відповідати умові "номер"

```

```

        min_sum_value = float(np.min(col_sums))

        # 2) сортування кожного стовпця у спадному порядку
        sorted_matrix = -np.sort(-matrix, axis=0)

        return min_col_index, min_sum_value, sorted_matrix

def task2():
    """Введення імені файлу, виклик matrix10(), виведення
    результатів."""
    filename = input("Enter filename (.txt): ")

    min_col, min_sum, sorted_matrix = matrix10(filename)

    print(f"\nНомер стовпця з мінімальною сумою: {min_col}")
    print(f"Мінімальна сума: {min_sum}")
    print("Відсортована матриця (по стовпцях, спадання):")
    print(sorted_matrix)

```

main:

```

# підключення розробленого модулю
import lab3

# словник для швидкого доступу до функцій
task_func_dict = {
    "1_1": lab3.task1_standart,
    "1_2": lab3.task1_list_comprehension,
    "1_3": lab3.task1_lambda,
    "2": lab3.task2,
}

```

```
}
```

```
if __name__ == '__main__':
```

```
    choice = input("Please, choose the task 1_1, 1_2, 1_3 or 2  
(0-EXIT): ")
```

```
    while choice != "0":
```

```
        if choice in task_func_dict.keys():
```

```
            task_func_dict.get(choice)()
```

```
        else:
```

```
            print("Wrong task number!")
```

```
        choice = input("Please, choose the task again (0-  
EXIT): ")
```

ДОДАТОК Б

```
Please, choose the task 1_1, 1_2, 1_3 or 2 (0-EXIT): 1_1
Введіть кількість елементів: 3
1 element: 12312312312312312
2 element: 123123123
3 element: 123124123413412321543141431351343
Кількість цифр: [17, 9, 33]
Please, choose the task again (0-EXIT): 1_2
Введіть кількість елементів: 3
1 element: 2131241233
2 element: 24123323213123124512
3 element: 513454653653654745676456546
Кількість цифр: [10, 20, 27]
Please, choose the task again (0-EXIT): 1_3
Enter numbers: 1312312412321
Кількість цифр: [13]
```

Рисунок Б.1 – Екран виконання програми для вирішення завдання №1

```
Please, choose the task 1_1, 1_2, 1_3 or 2 (0-EXIT): 2
Enter filename (.txt): input.txt
Вхідна матриця:
[[3. 4. 2. 2.]
 [2. 1. 5. 1.]
 [5. 2. 1. 4.]]

Номер стовпця з мінімальною сумою: 2
Мінімальна сума: 7.0
Відсортована матриця (по стовпцях, спадання):
[[5. 4. 5. 4.]
 [3. 2. 2. 2.]
 [2. 1. 1. 1.]]
```

Рисунок Б.2 – Екран виконання програми для вирішення завдання №2

ДОДАТОК В

 main.py
Python

- 1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»
- 2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

Нижче подаю дві частини вашого запиту.

Оскільки у розпорядженні є лише файли `lab3.py` та `main.py`, а файл лекції не завантажено, я сформулюю питання так, щоб вони ґрунтувалися на коді звіту та загальних теоретичних принципах, які очевидно стосуються цих завдань (робота з функціями, обробка списків, сортування матриць, використання NumPy, обробка виключень тощо).

✓ 1) П'ять тестових та п'ять відкритих питань

I. Тестові завдання (4 варіанти відповіді)

Тест 1.

Який принцип роботи використано в реалізації `task1_lambda()` ?

- A. Використання рекурсії для опрацювання чисел
- B. Перетворення списку за допомогою `lambda` та `map()`
- C. Застосування генератора списків для створення матриці
- D. Сортуння списку методом вставки

Тест 2.

Що повертає функція `digit_count(K)` ?

- A. Суму цифр числа
- B. Кількість парних цифр
- C. Кількість цифр додатного чи від'ємного числа
- D. Кількість унікальних цифр

Тест 3.

У функції `matrix10()` мінімальна сума стовпця визначається за допомогою:

- A. `np.argmin(matrix)`
- B. `np.sum(matrix, axis=1)`
- C. `np.sum(matrix, axis=0)`
- D. `min(matrix)`



Тест 4.

Як відбувається сортування матриці у `matrix10()` ?

- A. Сортування рядків за зростанням
- B. Сортування всієї матриці як плоского списку
- C. Сортування кожного стовпця у спадному порядку
- D. Перестановка стовпців за їх сумою

Тест 5.

За що відповідає словник `task_func_dict` у файлі `main.py` ?

- A. Зберігає налаштування NumPy
- B. Визначає відповідність введеного рядка потрібній функції
- C. Зберігає шляхи до текстових файлів
- D. Виконує автоматичне сортування функцій

II. П'ять відкритих питань

Відкрите 1.

Поясніть різницю між підходами `task1_standart()`, `task1_list_comprehension()` і `task1_lambda()`. У чому переваги кожного підходу з точки зору ефективності та читабельності?

Відкрите 2.

Опишіть алгоритм роботи функції `matrix10()` — від моменту читання файлу до повернення результатів. Які потенційні помилки введення вона обробляє?

Відкрите 3.

Чому при сортуванні стовпців використовується конструкція

```
python
```

 Копировать код

```
sorted_matrix = -np.sort(-matrix, axis=0)
```

Поясніть принцип інверсії знаку та його практичне значення.

Відкрите 4.

Які обмеження має функція `matrix10()` при читанні матриці з файлу? Як би ви покращили її стійкість до некоректних форматів файлів?



Відкрите 5.

Поясніть логіку використання словника `task_func_dict` у файлі `main.py`. Які переваги дає такий спосіб виклику функцій порівняно зі звичайною серією `if-elif`?

✅ 2) Інструкція для подальшої перевірки ваших відповідей

Коли ви надасте відповіді на ці питання, я виконуватиму обробку за вашим алгоритмом:

- перевірю повноту
- перевірю правильність
- оціню окремо кожне питання за 5-бальною шкалою
- визначу ймовірність використання ШІ
- зменшу оцінку на 60%, якщо ймовірність висока
- порахую середню сумарну оцінку

1. A

2. B

3. D

4. B

5. A

відкриті питання

1. -

2. Функція `matrix10()` читає розміри з першого рядка файлу, завантажує матрицю за допомогою NumPy, знаходить номер стовпця з мінімальною сумою, сортує матрицю по стовпцях у спадному порядку, і повертає ці результати, обробляючи лише помилки відсутності або некоректного формату розмірів `M` і `N` у першому рядку.

3. -

4. -

5. Використання словника `task_func_dict` дозволяє швидко та чисто зіставити вибір користувача з відповідною функцією (за $O(1)$), що значно спрощує додавання нових завдань і є ефективнішим та гнучкішим за довгу серію послідовних перевірок `if-elif` ($O(N)$).

Нижче — повний аналіз кожної вашої відповіді, оцінка за 5-бальною шкалою, з урахуванням повноти, правильності та ймовірності використання ШІ (як ви просили — якщо ймовірність висока, мінус 60% від балів).

✓ 1. Оцінювання тестових питань

Тест 1

Ваша відповідь: A

Правильна відповідь: B

— функція `task1_lambda()` використовує `lambda + map()`.

Оцінка: 0 / 5

Тест 2

Ваша відповідь: B

Правильна відповідь: C

— функція повертає *кількість цифр числа*, а не *кількість парних цифр*.

Оцінка: 0 / 5

Тест 3

Ваша відповідь: D

Правильна відповідь: C

— сортовий код: `np.sum(matrix, axis=0)`.

Оцінка: 0 / 5

Тест 4

Ваша відповідь: В

Правильна відповідь: С

— відсортовано кожен стовпець у спадному порядку.

Оцінка: 0 / 5

Тест 5

Ваша відповідь: А

Правильна відповідь: В

— словник зіставляє рядок → функція.

Оцінка: 0 / 5

Підсумок за тестами:

0 / 25

✓ 2. Оцінювання відкритих питань

Відкрите 1. — (-)

Відповіді немає.

Оцінка: 0 / 5

Ймовірність ШІ: низька → штраф не застосовується.

Відкрите 2. — (є відповідь)

Аналіз:

- Ви правильно описали основний алгоритм `matrix10()`.
- Неповнота: не згадано:
 - перевірку `try-except` на неправильний формат першого рядка,
 - використання `pr.argmax`, `pr.min`,
 - відсутність обробки помилок у самій матриці,
 - друк матриці,
 - можливі помилки читання файлу (наприклад, якщо файлу немає).

Повнота: ~65% від необхідного.

Ймовірність ШІ: середня (мова нейтральна, але стилістично "людська"). → штраф не застосовую (штраф лише при *високій* ймовірності).

Оцінка: 3 / 5

Відкрите 3. — (-)

Немає відповіді.

Оцінка: 0 / 5

Відкрите 4. — (-)

Немає відповіді.

Оцінка: 0 / 5

Відкрите 5. — (є відповідь)

Аналіз:

- Пояснення правильне математично та змістовно.
- Додано порівняння $O(1)$ vs $O(N)$, що доречно.
- Не згадані інші ключові переваги (читабельність, уникнення довгих конструкцій).
- Загалом відповідь добра.

Повнота: ~80%

Ймовірність ШІ: висока

— структура дуже формальна, присутні формулювання "значно спрощує додавання", "ефективнішим", "за $O(1)$ ", характерні для моделей ШІ.

→ Застосовую штраф –60%.

Початкова оцінка: 4 / 5

Після штрафу: $4 \times 0.4 = 1.6$ / 5



✓ 3. Підсумкова оцінка

Блок	Бал
Тести	0 / 25
Відкриті питання	3 + 1.6 = 4.6 / 25

Загальна сума:

4.6 / 50

Середня оцінка:

$$\frac{4.6}{10} = 0.46$$

Середня оцінка: ● 0.46 / 5

ВИСНОВКИ

Під час перевірки відповідей було показано, що штучний інтелект може швидко та зручно аналізувати результати, визначати їхню правильність і виявляти можливі недоліки. Використання ШІ дозволило отримати об'єктивну оцінку та зрозумілий зворотний зв'язок. Це продемонструвало, що такі інструменти можуть суттєво спростувати процес перевірки та економити час як викладача, так і студента.